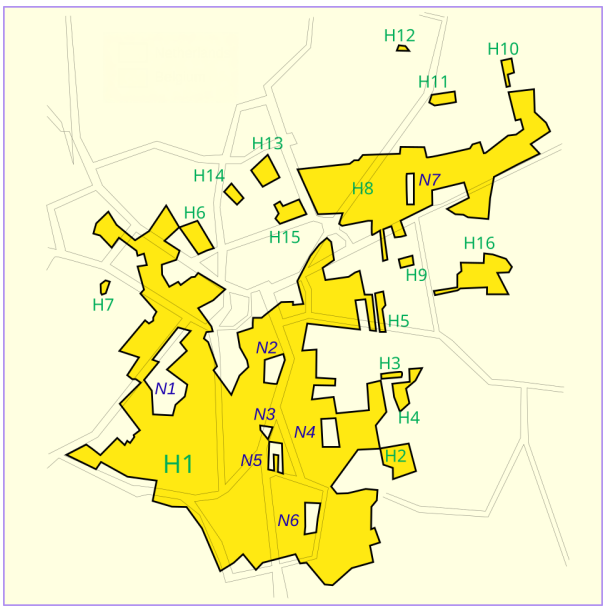
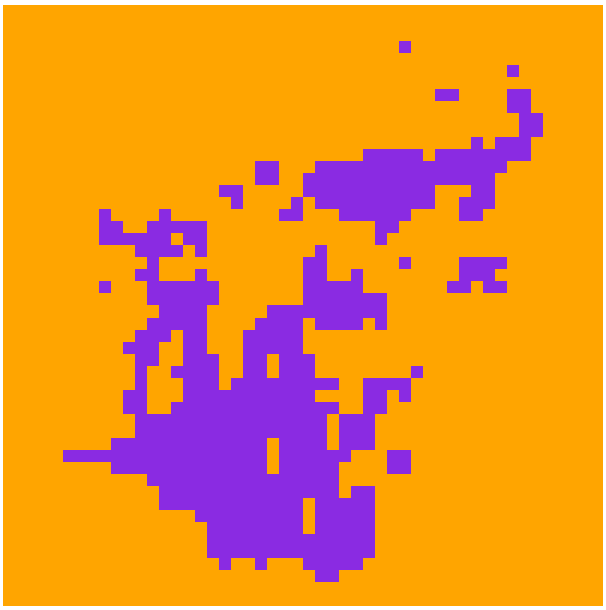
1 Belgium Netherlands Border [40 Marks]



(a) The Baarle-Nassau border



(b) Pixelated input

1.1 Image Processing and Dataset Creation [5]

The border between Netherlands and Belgium at Baarle-Nassau is a complex one. Your job here is to best model the border using an MLP. Provided is a 50 by 50 pixelated version of the border image where pixels are colored to represent two countries: orange for Netherlands and purple for Belgium.

1. **Binary Mask Conversion:** Convert the input image into a 0-1 binary mask.
2. **Dataset Class:** Create a dataset class with a function that randomly returns all the pixels from the image in a shuffled order. The format for each returned sample should be: $((x, y), L)$, where:
   - (x, y) are the pixel coordinates, normalized to the range $[0, 1]$.
   - L is the corresponding label, where $L \in \{0, 1\}$.

1.2 Neural Network Implementation from Scratch [15]

Core Components

1. **Linear Layer Class:** Implement a 'Linear' layer class that takes input width, output width, and an activation function as arguments. It must store all data required for both forward and backward passes, including:
   - Forward pass data: weights, biases.

- Backward pass data: layer outputs, cumulative gradients.

2. **Activation Function Classes:** Implement each of the following activation functions as a separate class. Each class must contain methods for both the forward pass (calculating the activation) and the backward pass (calculating the derivative).

   - ReLU
   - Tanh
   - Sigmoid
   - Identity (no activation)

3. **Model Class:** Implement a 'Model' class that accepts a list of 'Linear' layers and a loss function type.

   - The model must implement 'forward' and 'backward' pass functions that sequentially execute the corresponding methods in each layer.
   - For now, implement the Mean Squared Error (MSE), and Binary Cross Entropy (BCE) loss function.

   Additionally, implement the following methods in the 'Model' class:

   - `train(x, y)`: Performs a forward pass on the input 'x' to get a prediction y_{pred}. It then computes the loss between y_{pred} and the true label 'y', backpropagates the loss, and adds the resulting gradients to the cumulative gradient stored in each layer. **Do not update the parameters in this step.**
   - `zero_grad()`: Resets the cumulative gradient in each layer to zero.
   - `update()`: Updates all model parameters using the values in the cumulative gradient, and then calls `zero_grad()`.
   - `predict(x)`: Performs a forward pass on the input 'x' and returns the prediction y_{pred}.
   - `save_to(path)` and `load_from(path)`: These methods should save/load all model parameters to/from a file in `.npz` format. Loading should throw an error if the architecture of the current model (i.e., the shapes of weights and biases arrays) does not match the shapes of the arrays in the saved file.

4. **Training Procedure:** Implement a training loop that takes `batch_size` and `grad_accumulation_steps` as parameters.

   - For each batch, call the `train()` method.
   - After `grad_accumulation_steps` have been completed, call the `update()` method to update the parameters.
   - Keep track of the loss at each iteration. Log the training progress (e.g., using `tqdm`).

- Upon completion of training, save both a plot of the training loss versus the number of samples seen and the final model parameters to a unique run folder (e.g., `runs/{timestamp}`).
- Save all the run details, include hyperparameters, losses, accuracies to WandB.
- **Note on Plotting:** When plotting the loss vs. samples graph, each data point a model trains on is considered a single sample. Ensure the x-axis is adjusted correctly by factoring in `batch_size` and `grad_accumulation_steps` to allow for consistent comparisons between runs with different hyperparameters.

5. **Early Stopping:** Implement an early stopping mechanism. As there is no separate validation set, training should stop when the loss drops by less than 1% over the last 10 epochs. Formally, stop if $L_t \geq 0.99 \cdot L_{t-10}$. The parameters `patience=10` and `relative_loss_threshold=0.01` should be configurable.

1.3 Sanity Check [5]

1.3.1 The XOR Problem [2]

Before applying the model to the map data, test its correctness on a simple dataset.

1. Create a dataset for the XOR function: $\{0,1\} \times \{0,1\} \to \{0,1\}$. The model should predict $x_1 \oplus x_2$.
2. Train and test the model with a variety of architectures. Vary the number of layers, the width of each layer, and use all implemented activation functions.
3. Ensure that the model converges to 100% accuracy for all training runs.

1.3.2 Gradient Approximation [3]

When running the above, also verify the gradients are being computed correctly. Create a function to compute the per-weight gradient :

$$\nabla_{\theta_i} L(x;\theta) = \frac{L(x;\theta + \epsilon \cdot \hat{\theta_i}) - L(x;\theta - \epsilon \cdot \hat{\theta_i})}{2\epsilon}$$

And verify that the obtained values converge to $\nabla_\theta L(x;\theta)$ obtained via backpropagation. (For small ϵ)

1.4 Map Prediction and Analysis [5]

Use the implemented neural network to predict whether each pixel in the map belongs to France or Belgium. The input is the 2D coordinate (x, y), and the label is $L \in \{0, 1\}$.

1. Train multiple model architectures (width of layer, number of layers, activation functions) until convergence (as determined by early stopping). Compare the final loss and accuracy of the predicted maps. Accuracy is defined as the proportion of correctly classified pixels.

2. For each run, save the following images side-by-side in a single plot within the run folder:
    - The ground-truth map.
    - The map generated from the model's predictions.
    - An error map showing misclassified pixels overlaid on the ground-truth image.
3. **Experimentation with Architecture:**
    - For a fixed layer width, vary the number of layers. Plot the final loss and accuracy as a function of depth.
    - For a fixed number of layers, vary the width of the layers. Plot the final loss and accuracy as a function of width.
4. **Experimentation with Hyperparameters:**
    - Train with a variety of `batch_size`, `grad_accumulation_steps`, and learning rates.
    - Provide a comparison of the time taken to converge and the total number of samples needed to converge for different hyperparameter settings.

1.5 Final Challenge [10]

Based on the insights from the previous experiments, attempt to achieve a target accuracy of 91% .

1. **Goal 1: Minimize Model Size.** Your first goal is to achieve the target accuracy with the minimum possible number of parameters. Comment on the process and reasoning that led you to your optimal architecture.
2. **Goal 2: Minimize Training Samples.** Your second goal is to achieve the target accuracy using the minimum number of training samples to converge. Comment on how you arrived at the optimal training parameters (learning rate, batch size, etc.).

Note: You may apply multiple training methods with checkpointing to achieve optimal results. Experiment and record your methodology.